HARP: HUMAN VS. AI RECORDINGS PREDICTOR

Andrew Meaux, Christopher Leinweber, & Taylor Chambers
Department of Computer Science
Rice University
{apm7, cl158, tbc6}@rice.edu

ABSTRACT

With recent advances in AI models that can generate various forms of media, companies and individuals are interested in being able to detect whether a given sample is human-generated or model-generated. In particular, several groups have recently released advanced models capable of generating coherent, long-form music in raw audio form. With an aim to avoid an influx of content generated by such models that overwhelms digital distributors, we introduce `HARP`*, a binary classification model that predicts whether an input music recording is human- or AI-generated.*

1 INTRODUCTION

1.1 BACKGROUND

Advancements in generative AI have raised mainstream concerns in use cases such as video deep fakes or spoofed human speech. Beyond text, imagery, and speech, generative AI has now been extended into music. State-of-the-art music generation models include OpenAI's Jukebox (Dhariwal et al., 2020), Google's MusicLM (Agostinelli et al., 2023), and Meta's MusicGen (Copet et al., 2023). Just as the online image community has been impacted by the appearance of AI generated images, these new music models have resulted in legitimate music sources being flooded with AI generated music. Just recently, two of the top streaming services Apple Music and Spotify removed AI-originated content from their library (Ngila, 2023). Additionally, mainstream recording artists have begun to express disdain for AI generated music and issued a call to action for government regulation (Genovese, 2023). For this reason, our team proposes a machine learning model that can successfully discern human-generated music from AI-generated music. We hypothesize that the application of existing techniques used to detect AI generated speech can be successful in detecting AI generated music as well. One such solution is Dessa's Deepfake Audio Detection, a system which discerns real speech from generative speech by analyzing visual representations of the audio, known as a spectrogram, in a neural network using temporal convolutions (Dessa, 2020). A spectrogram is a visual representation of an audio file, where the audio pitch and amplitude are plotted over time. Our team plans to implement a similar technique, and more, in the space of AI generated music.

1.2 OBJECTIVES

Our objective is to develop a machine learning model that can successfully classify an audio file containing music as being either `HUMAN-GENERATED` or `AI-GENERATED`. The definitions of these two classes are as follows:

- `HUMAN-GENERATED`: The audio file, as indicated by the mel spectrogram, is likely an original work derived entirely through human creation.
- `AI-GENERATED`: The audio file, as indicated by the mel spectrogram, is likely created using generative AI.

2 RELATED WORKS

Our objective is to identify whether a music file is generated by human or AI. This naturally leads to a binary classification task, where each file is classified into one of the two categories. Since we

intend to perform the classification using an image representation of the raw audio (specifically the mel spectrogram), we explored machine learning architectures that are specifically designed to work with image inputs. We compared the performance of two machine learning models: ResNet152, a convolutional neural network (CNN), and Vision Transformer (ViT), a modification of the Transformer architecture.

2.1 Convolutional Neural Networks

A convolutional neural network is so named because it includes convolutional layers. These layers differ from the traditional fully-connected layers in that they apply smaller filters (or kernels) that slide along the input data to learn features. Each kernel learns a different feature from the input, and converts the input data into a feature map. Convolutional layers also include an activation function (often ReLU or tanh) to introduce non-linearity to the output. These feature maps are often downsampled using pooling techniques to reduce the dimensionality of the data and to increase the robustness of the learned features. Pooling applies a small filter, such as 2x2, where the values within each patch of that size are averaged (average pooling) or the maximum is selected (max pooling). The convolution-activation-pooling process is generally repeated a number of times. Final classification is done via one or more fully connected layers. The shared weights of the convolutional layers massively reduce the number of connections when compared to a fully connected layer, allowing for deeper networks with larger input sizes, a necessity for working with images.

While the basic architecture of CNNs was proposed decades earlier, one of the first applications of CNNs to image classification was a 3-layer network (LeNet 1) built to recognize hand-written ZIP code numbers by LeCun et al. (1989). The architecture of this model is shown in Figure 1 below. While pioneering at the time, the model is limited in that it is only designed to receive inputs of size 28x28. It was also slow and not popular at the time, as GPUs and the associated software for rapid matrix calculations were not yet available. The next significant advancement was a 7-layer network (LeNet 5), also to recognize hand-written digits, also by Lecun et al. (1995). The more advanced model was required due to the release of the MNIST dataset, for which LeNet 1 was not powerful enough. This progression illustrates one of the fundamental principles of CNNs—processing more complex images often requires deeper networks with larger numbers of parameters.

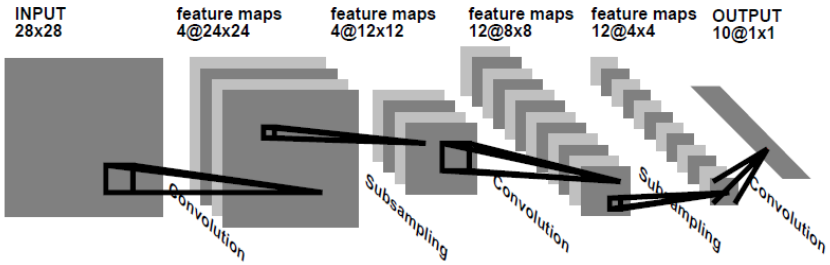

Figure 1: Architecture of LeNet 1 model, reproduced from Lecun et al. (1995)

Computational requirements meant that CNNs did not advance significantly for a number of decades, before rapidly accelerating in the 2010s. In 2012, Krizhevsky et al. (2012) proposed a much larger and moderately deeper CNN (AlexNet) that also included much larger fully connected layers at the output, demonstrating a significant leap in performance in the ImageNet Large Scale Visual Recognition Challenge. It was the first model to achieve a sub-25% error rate on the ImageNet Challenge. The model was also interesting in that it split the computation across two GPUs to increase speed, with only certain layers communicating between GPUs. The architecture of AlexNet is shown in Figure 2 below.

In 2015, He et al. (2015) at Microsoft proposed a significantly deeper CNN (ResNet) than anything previously established, demonstrating another significant jump in performance. They did so by introducing residual or "skip" connections, which allowed for a significant increase in depth while avoiding the normal accompanying degradation and vanishing gradient problems. Residual networks group the convolutional layers into "building blocks". Inputs (and backpropagated gradients) follow a standard path through each layer of the block, but a parallel shortcut is introduced. The

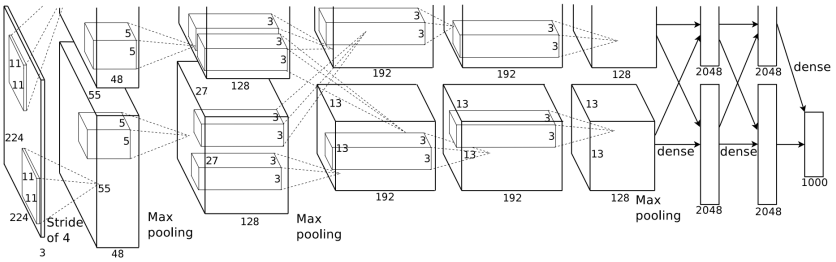

Figure 2: Architecture of AlexNet model, reproduced from Krizhevsky et al. (2012)

input is allowed to skip the layers of the block, being added directly to the output of the block. This adds minimal computational complexity to the network, but in practice markedly improves the performance of the network. The architecture of a 34-layer ResNet is shown in Figure 3. The bottom model is a "plain" 34-layer architecture, and the top model adds the residual connections. Note that the remarkable depth makes it difficult to visualize even this structure, among the smallest versions of ResNet. The available pre-trained architectures include models with 101 and even 152 layers, and the ResNet structure can be generalized to any number of layers.

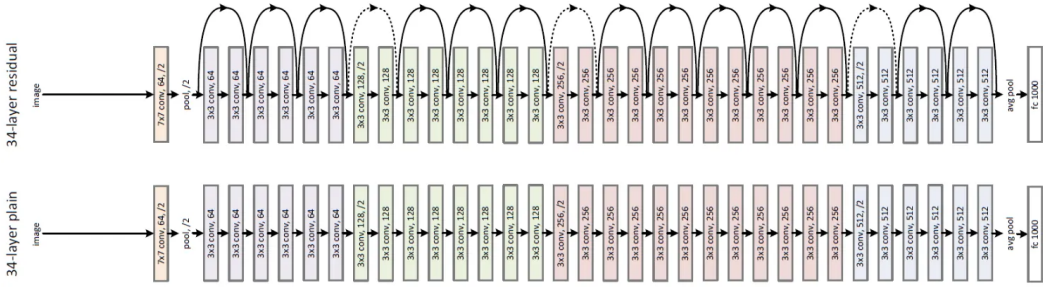

Figure 3: Architecture of ResNet-34 model, reproduced from He et al. (2015)

Historically, very deep neural networks were known to suffer from the vanishing gradient problem. Earlier activation functions such as sigmoid and tanh forced the gradient values into a small range around zero, so repeated applications of the activation function across layers drove the gradients towards zero. Additionally, the gradients are multiplied layer by layer, causing them to diminish exponentially with depth. This had previously been somewhat mitigated by the use of ReLU activation functions, which do not shrink positive gradient values. Residual connections further mitigate vanishing gradients by allowing gradients to skip some layers, thereby preventing some of the forcing towards zero.

Deep neural networks can also suffer from the degradation problem, where adding additional layers decreases the performance of the model. This is due to the increased complexity of the model, which makes optimization less efficient during training and often increases the training error and slows convergence. Residual connections have been empirically shown to mitigate this problem.

Recently, Dessa (2020) has applied these powerful deep CNNs to the detection of speech audio deepfakes. Their application was for the detection of computer-generated speech, but our team believes that the same technique can be applied to detect AI-generated music. Their model begins by converting audio files to mel spectrograms, which serve as the input to the model. Many convolutional layers (the number and size are a model input) are applied over the time dimension of the spectrogram, along with masked pooling to prevent overfitting. The results of the convolutional layers are then passed to a dense layer and finally a sigmoid activation function, resulting in a predicted probability of the audio sample being real. As discussed above, CNNs are designed to extract features from images. While human-generated and AI-generated sound may sometimes be indistinguishable to the human ear, there are minute differences in their spectrograms. With appropriate training, the CNN is able to pick up these differences visually.

2.2 VISION TRANSFORMER

In 2017, Vaswani et al. (2017) revolutionized the field of Natural Language Processing with the introduction of the Transformer in the seminal paper "Attention is all you need". The Transformer replaced the previous state of the art gated recurrent neural networks (RNNs), including gated recurrent units (GRUs) and long short-term memory networks (LSTMs) for machine translation and other language tasks. As opposed to the recurrent input processing of RNNs, the Transformer uses an attention mechanism to process all input tokens in parallel. This allowed for a massive increase in training speed due to its highly parallelizable process, which is well tailored for GPU calculations. It also proved highly suitable for machine translation, since the attention mechanism allows the output tokens to learn from whichever input tokens are more relevant, not necessarily in sequential order.

In 2020, Dosovitskiy et al. (2020) applied the Transformer architecture to computer vision with the introduction of Vision Transformer (ViT). ViT pre-processes images by dividing them into fixed-size non-overlapping patches, e.g. 8x8 pixels, which are then flattened into a linear vector. This becomes the equivalent of an input token to a traditional Transformer. To mimic the positional awareness inherent to CNNs, the patch tokens also have embedded positional information. The patch tokens with positions are then fed through a series of Transformer encoder layers. Finally, the embedding of the first token is passed through a feed-forward neural network to provide a final classification result. The architecture of ViT is shown in Figure 4.

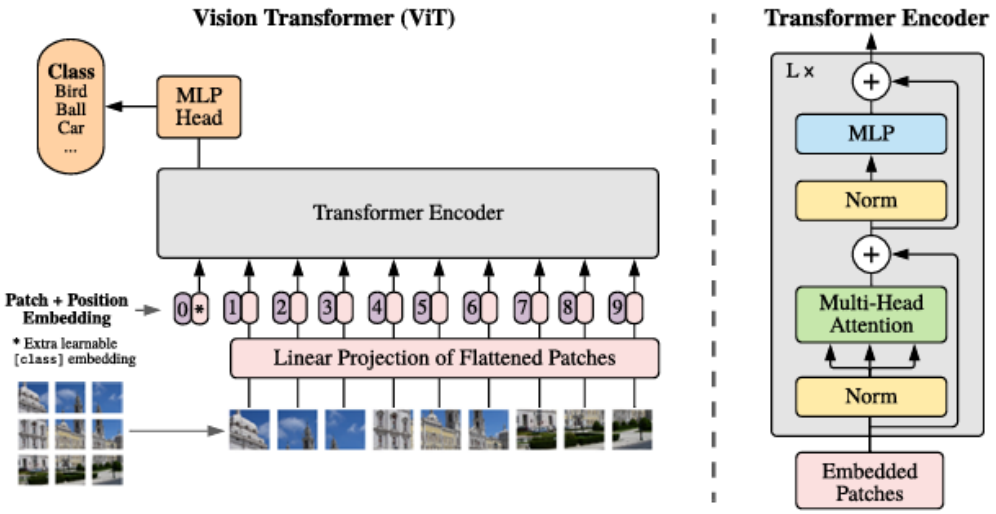

Figure 4: Architecture of Vision Transformer model, reproduced from Dosovitskiy et al. (2020)

ViT has certain advantages and disadvantages when compared to state of the art CNNs. Notably, ViT models are generally more parameter efficient than CNNs, meaning that they can often represent more complex features than CNNs given the same number of parameters. They are also better able to capture long-range patterns in the data than CNNs (except for very deep CNNs), since their attention mechanism can capture broad dependencies in a single layer. CNNs have local receptive fields as a fundamental part of their structure, due to their use of small kernels to learn features. ViTs are often more computationally complex than CNNs due to their attention mechanism, and they often require significantly more data and training to reach optimal performance due to that added complexity. For cases where sufficient amounts of data and training time are available, ViTs can achieve state of the art performance on image classification.

3 DATA DESCRIPTION

The most common representation of raw digital audio is an encoding called Pulse Code Modulation (PCM). In summary, PCM involves sampling a continuous analog signal at consistent time intervals, such as once every $1/12,000$ of a second. The inverse of this sampling period is then the sampling

rate, in this case 12 kHz. Due to the Nyquist-Shannon-Kotelnikov sampling theorem, the maximum frequency that can be represented is half the sampling rate. Since the upper limit of human hearing is roughly 20 kHz, this leads to typical sampling rates slightly larger than 40 kHz, such as 44.1 kHz.

The magnitude of the sampled signal (e.g. the voltage in a microphone circuit) is then quantized into a discrete number of values, dependent on the number of bits dedicated to storing each sample, as shown in Figure 5. For example, with a bit depth of 8 bits, there are then $2^8 = 256$ discrete values the voltage could be quantized to. Finally, PCM is extensible with the number of audio channels it supports, such as a left channel and a right channel for stereo. The samples of each channel are simply interleaved in a single PCM stream.

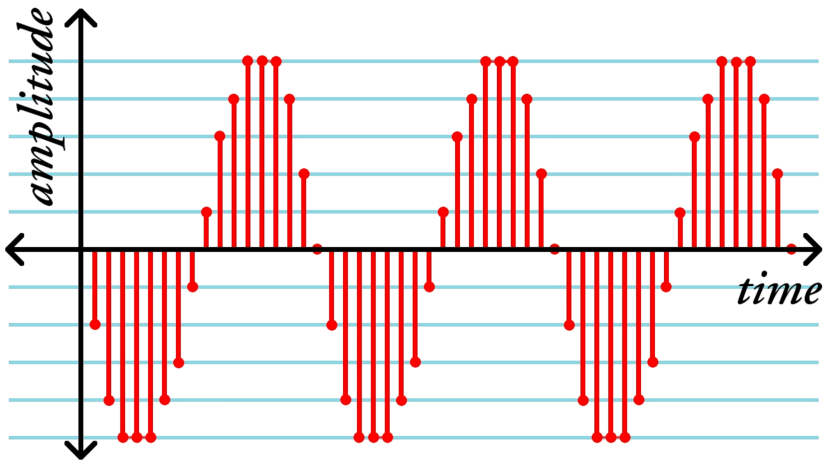

Figure 5: Quantized samples taken at consistent intervals in PCM, reproduced from Montgomery (2010)

Therefore, PCM encoding can be described in three parameters: the sampling rate, the bit depth, and number of channels. For example, Compact Disc (CD) audio utilizes PCM with a sampling rate of 44.1 kHz, a bit depth of 16 bits, and 2-channel audio for stereo. For the purposes of our dataset, we do not give any extra consideration to whether data samples have been further encoded in a lossy compression scheme such as MPEG-1 Audio Layer III (MP3), as such files must be decoded to raw PCM for any playback or processing. As such, the focus for our data sources are any differences in the three aforementioned PCM parameters.

For visualizing raw digital audio, there are generally two methods: time-domain and frequency-domain. For time-domain visualization, the amplitude of the signal is plotted against time to display a waveform. The amplitude is generally expressed relative to the maximum quantized value (otherwise known as full-scale). For frequency-domain visualization, many Short-Time Fourier Transforms (STFT) are taken over a sequence of windows across the original signal, to acquire the spectrum of each windowed segment:

$$\mathbf{STFT}(x, W, \tau, \omega) = \int_{-\infty}^{\infty} x(t) W(t - \tau) e^{-i\omega t} dt \tag{1}$$

Where $x(t)$ is the signal and $W(\tau)$ is a window function. The power of each spectra is then plotted as a color map over time, to see the intensity of each audio frequency as it changes over time. This is known as a spectrogram, which can reveal details about the underlying audio. For example: thin, short-duration lines that span the entire frequency bandwidth are often percussion elements such as a snare drum. Figure 6 demonstrates this by plotting the time-domain waveform as well as the spectrogram of demo audio. Intended to help develop an intuition for the spectrogram, the demo audio consists of a pure 440 Hz (note A4) tone, followed by two bass drum hits, two snare drum hits, two hi-hat cymbal hits, and then the note A4 played on a piano. The frequency axis of the

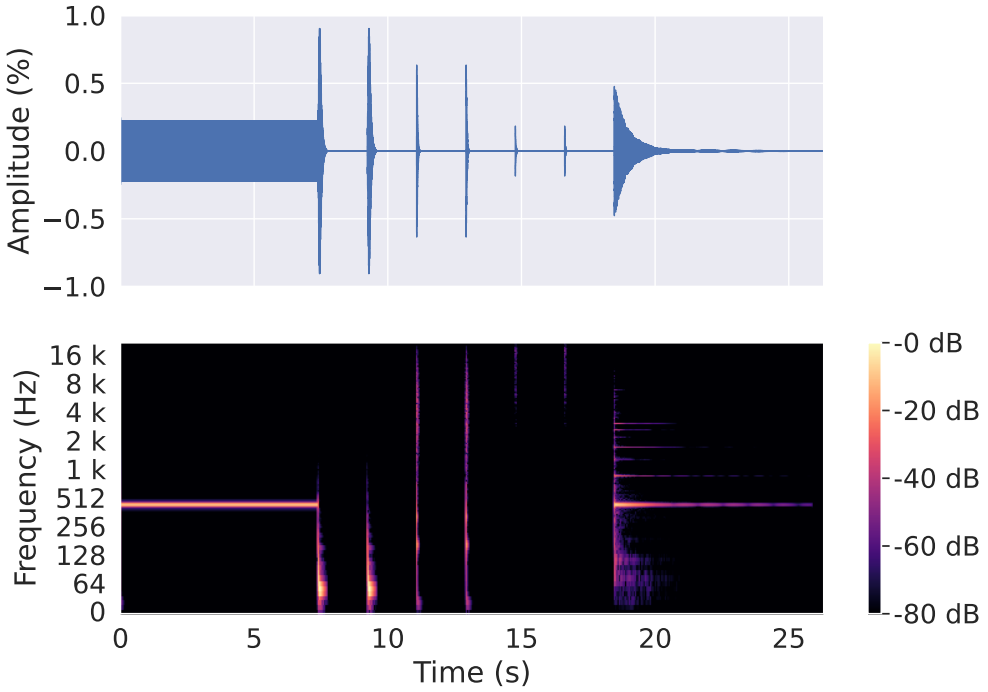

Figure 6: Time-domain waveform and frequency-domain spectrogram of demo audio

spectrogram is plotted on a $\log_2$ scale so that octaves are evenly spaced. Figure 7 shows a similar plot, but of an actual song clip.

Since the task for HARP is to classify a given music recording as either human-generated or AI-generated, this naturally lends itself to a dataset comprising two distinct parts:

1. Audio files labeled as human-generated, and
2. Audio files labeled as AI-generated.

For item 1, we discuss a source of Creative Commons-licensed music recordings, namely the Free Music Archive (FMA), in Section 3.1. For item 2, we discuss scraping the demo websites for various AI Music Generation methods, as well as utilizing publicly-released models with weights to generate further examples in Section 3.2.

3.1 Free Music Archive Dataset

To acquire audio labeled as human-generated for our overall dataset, we utilized the Free Music Archive (FMA) dataset. The FMA dataset was created by Defferrard et al. (2017) to provide a standard dataset for music information retrieval (MIR) tasks. The intent for FMA is to be for music what ImageNet is for images. The full FMA dataset comprises 106,574 Creative Commons-licensed music tracks from 16,341 artists. Beyond the full version, the FMA dataset is available in four tiers as described in Table 1. Instructions for downloading the dataset are available on the FMA GitHub[1].

For HARP, we used the FMA small tier, which consists of 8,000 genre-balanced tracks. All tracks within the dataset are MP3-encoded with a sampling rate of 44.1 kHz, average bit rate of 263 kilobits per second, and two channels (stereo). To demonstrate an example entry in this dataset, the audio from a single example track is plotted in both the time domain and frequency domain in Figure 7. To provide further typical information about the files contained in the FMA dataset, a subset of the

[1] https://github.com/mdeff/fma

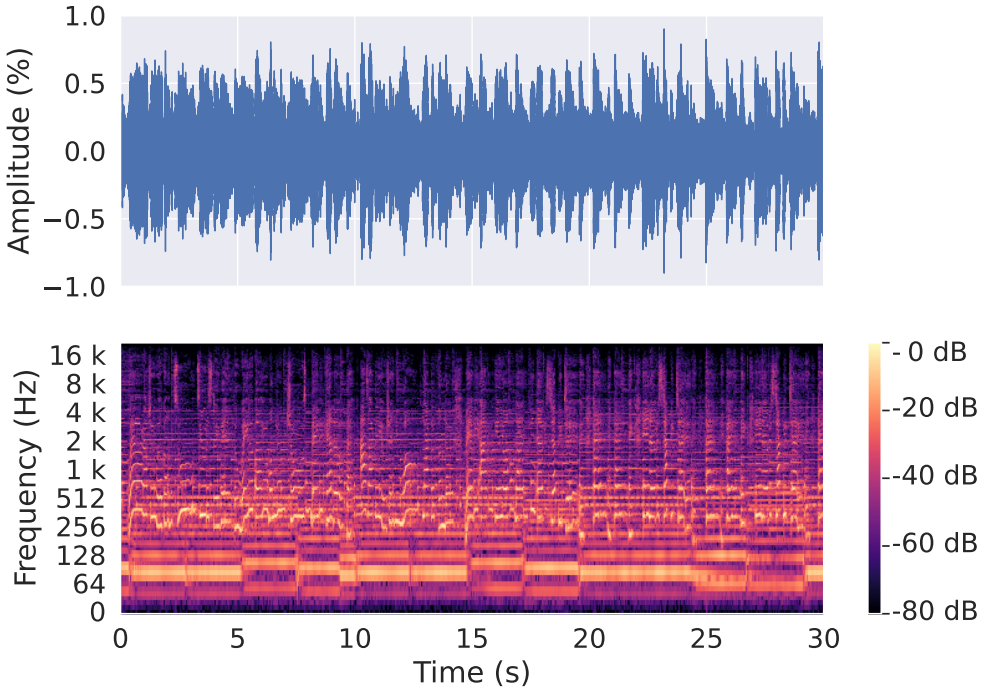

Figure 7: Time-domain waveform and frequency-domain spectrogram of music

Table 1: Tiers of the Free Music Archive

tier	audio files	genres	length per audio file (s)	total size (GiB)
small	8,000	8	30	7.4
medium	25,000	16	30	23
large	106,574	161	30	98
full	106,574	161	278	917

MediaInfo[2] output for the same file is reproduced in Listing 1. The full track can be listened to on the FMA website[3].

[2] https://mediaarea.net/en/MediaInfo

[3] https://freemusicarchive.org/music/Wildlight/The_Tide_Acoustic/Wildlight_-_The_Tide_Acoustic_-_07_Move_Like_The_Ocean_Acoustic

Table 2: Samples of AI-generated music

Model	Number of audio files
MusicGen	5,540
MusicLM	219
Noise2Music	119
Jukebox	7,100
Moûsai	67
Riffusion	28
Total	13,073

```
General
CompleteName                     : 144552.mp3
Format/String                    : MPEG Audio
FileSize/String                  : 1.15 MiB
Duration/String                  : 30 s 14 ms
Album                            : The Tide (Acoustic)
Album/Performer                  : Wildlight
Track                            : Move Like The Ocean (Acoustic)


Audio
Format/String                    : MPEG Audio
Format_Profile                   : Layer 3
Duration/String                  : 30 s 15 ms
BitRate_Mode/String              : Constant
BitRate/String                   : 320 kb/s
Channel(s)/String                : 2 channels
SamplingRate/String              : 44.1 kHz
Encoded_Library/String           : LAME3.99r
```

Listing 1: MediaInfo output for FMA track ID #144552

A limitation of the FMA dataset of any tier is that the creators make no claim of the dataset being representative of all music.

3.2 AI-GENERATED MUSIC

To acquire audio labeled as AI-generated for our overall dataset, we utilized samples that are known to be created by the following music generation models:

1. MusicGen
2. MusicLM
3. Noise2Music
4. Jukebox
5. Moûsai
6. Riffusion

To gather known AI-generated samples, our team either scraped the samples from the model's demonstration website, or directly generated samples using pretrained models. Table 2 summarizes the amount of samples from each data source to be labeled as AI-generated.

3.2.1 MUSICGEN

Copet et al. (2023) from Meta provide MusicGen, which is a language model that, rather than utilizing text tokens, operates over tokens that represent the latent space of high fidelity audio recordings:

in particular the encoded tokens of the Encodec neural music codec from Défossez et al. (2022). Input text descriptions are passed through a text encoder model to obtain a vector representation. This representation is then passed to MusicGen, which is trained to predict audio tokens, a.k.a. audio codecs, from the encoded text. The predicted audio tokens are then decoded using the Encodec audio compression model to generate an audio waveform. An audio sample can also be used as a prompt, in which case it is encoded using Encodec and this representation used to predict the output audio.

The demonstration website[4] for MusicGen contained 19 samples that were scraped. In addition, Meta provides several pre-trained MusicGen models with a simple Python interface via their Audiocraft GitHub[5]. Using the "large" text-to-music pre-trained model with 3.3B parameters, our team created 5,521 additional MusicGen samples. The text used to condition each of these additional samples was from the 5,521 captions written by professional musicians in the MusicCaps[6] dataset from Agostinelli et al. (2023).

A representative MediaInfo output from one of the samples generated by our team is shown in Listing 2. Of note is the duration of 30 seconds, the format of raw PCM in a `.wav` container, a sampling rate of 32 kHz, and a single mono channel. The 19 samples scraped from the MusicGen website have the same properties, except being MP3-encoded at a constant 256 kilobits per second.

```
General
Complete name                            : 0.wav
Format                                   : Wave
Format settings                          : PcmWaveformat
File size                                : 1.83 MiB
Duration                                 : 30 s 0 ms
Overall bit rate mode                    : Constant
Overall bit rate                         : 512 kb/s
Writing application                      : Lavf59.27.100

Audio
Format                                   : PCM
Format settings                          : Little / Signed
Duration                                 : 30 s 0 ms
Bit rate mode                            : Constant
Bit rate                                 : 512 kb/s
Channel(s)                               : 1 channel
Sampling rate                            : 32.0 kHz
Bit depth                                : 16 bits
```

Listing 2: MediaInfo output for audio created using MusicGen

According to Copet et al. (2023), MusicGen generates music which is plausible, has strong semantic alignment to the prompting text, and shares broad similarity with prompting reference music. According to subjective human ratings, MusicGen outperforms prior models on both overall quality and relevance to input prompts. Subjectively according to this paper's authors, MusicGen produces the highest-quality, most natural-sounding music.

3.2.2 MusicLM

Agostinelli et al. (2023) provide MusicLM, a model very similar to MusicGen, except hierarchical in nature and utilizing the SoundStream neural codec from Zeghidour et al. (2021). Like MusicGen, MusicLM can be conditioned on either a text or audio prompt and uses a Transformer-based encoder to project the input into an embedding space. This embedding is then decoded to generate music. Unlike MusicGen, MusicLM uses hierarchical processing to first generate semantic tokens, then acoustic tokens based off of both the embedded input and the semantic tokens. MusicLM subjectively and objectively outperforms prior models such as Riffusion and Mubert.

[4] https://ai.honu.io/papers/musicgen/
[5] https://github.com/facebookresearch/audiocraft
[6] https://research.google/resources/datasets/musiccaps/

The demonstration website for MusicLM[7] contained 150 samples that were scraped. In addition, the MusicGen website contained an additional 15 MusicLM samples that served as a comparison. Beyond the pre-generated samples, Google provides public access to an unspecified version of MusicLM via their AI Test Kitchen[8]. On this website, a user can provide a text prompt and MusicLM will generate two 20-second music samples conditioned on that text. Using this tool, along with 54 music descriptions randomly selected from the aforementioned MusicCaps dataset, our team created additional MusicGen samples.

A representative MediaInfo output from one of the MusicLM samples generated by our team is shown is Listing 3. Of note is the duration of 20 seconds, the sampling rate of 24 kHz, and the singular mono channel.

```
General
Complete name                            :
    AI_Test_Kitchen_this_is_a_hindustani_classical_music_piece.mp3
Format                                   : MPEG Audio
File size                                : 310 KiB
Duration                                 : 19 s 848 ms
Overall bit rate mode                    : Constant
Overall bit rate                         : 128 kb/s
Writing library                          : LAME3.100

Audio
Format                                   : MPEG Audio
Format version                           : Version 2
Format profile                           : Layer 3
Duration                                 : 19 s 848 ms
Bit rate mode                            : Constant
Bit rate                                 : 128 kb/s
Channel(s)                               : 1 channel
Sampling rate                            : 24.0 kHz
Compression mode                         : Lossy
```

Listing 3: MediaInfo output for audio created using MusicLM

3.2.3 NOISE2MUSIC

Huang et al. (2023) provide Noise2Music, which generates 30-second music clips from a text prompt using a series of diffusion models, as opposed to the previously mentioned methods of a language model operating on neural codec tokens. Noise2Music consists of two variations. The first variation, the waveform model, utilizes low sampling-rate audio as an intermediate representation before cascading into further diffusion models to ultimately produce 24 kHz audio. The second variation, the spectrogram model, instead first generates a log-mel spectrogram conditioned on the prompt before cascading through further diffusion models.

Our team scraped 119 samples from the demonstration website for Noise2Music[9]. A representative MediaInfo output from one of the samples is shown in Listing 4. Of note is the duration of 20 seconds, the sampling rate of 24 kHz, and the singular mono channel.

[7] https://google-research.github.io/seanet/musiclm/examples/
[8] https://aitestkitchen.withgoogle.com/experiments/music-lm
[9] https://google-research.github.io/noise2music/

```
General
Complete name                            : genre_0_wavegen_0.wav
Format                                   : Wave
Format settings                          : PcmWaveformat
File size                                : 1.35 MiB
Duration                                 : 29 s 440 ms
Overall bit rate mode                    : Constant
Overall bit rate                         : 384 kb/s

Audio
Format                                   : PCM
Format settings                          : Little / Signed
Codec ID                                 : 1
Duration                                 : 29 s 440 ms
Bit rate mode                            : Constant
Bit rate                                 : 384 kb/s
Channel(s)                               : 1 channel
Sampling rate                            : 24.0 kHz
Bit depth                                : 16 bits
```

Listing 4: MediaInfo output for audio created using Noise2Music

Based on objective measures, Noise2Music outperforms all prior models, including Riffusion, Mubert, and MusicLM in terms of audio quality.

3.2.4 JUKEBOX

Dhariwal et al. (2020) from OpenAI provide Jukebox, which uses autoregressive Transformers on Vector-Quantized Variational AutoEncoder (VQ-VAE) tokens to generate music audio (van den Oord et al., 2018). To work with a compressed latent representation of raw audio, Jukebox uses VQ-VAE, but trained on raw audio waveforms and with additional tweaks, such as a short-time fourier transform (STFT) component to the loss function. Within this latent space, Jukebox uses an autoregressive Sparse Transformer (Child et al., 2019).

Given a genre, an artist, and optional lyrics, Jukebox can generate novel CD-quality audio files at a 44.1 kHz sampling rate. The best results are obtained with a 1 to 4 minute length, and the output retains thematic consistency throughout the duration. The audio generation is conditioned based on whether or not the supplied genre or artist falls within Jukebox's training set (Dhariwal et al., 2020).

Our team scraped 7,100 samples from SoundCloud[10] that were hosted as part of OpenAI's official Jukebox demos. These samples are generated via Jukebox's top performing 5B parameter model and include both lyrical and non-lyrical audio files. A representative MediaInfo output from one of the samples is shown in Listing 5. Of note is the duration of more than a minute, the sampling rate of 44.1 kHz, and the 2 channels.

[10] https://soundcloud.com/openai_audio

```
General
Complete name                            : 0.5 Classic Pop + 0.5 Country, in the style of 0.5 Frank Sinatra + 0.5 Alan Jackson - Jukebox-796469212.mp3
Format                                   : MPEG Audio
File size                                : 1.09 MiB
Duration                                 : 1 min 11 s
Overall bit rate mode                    : Constant
Overall bit rate                         : 128 kb/s
Writing library                          : LAME3.100

Audio
Format                                   : MPEG Audio
Format version                           : Version 1
Format profile                           : Layer 3
Format settings                          : Joint stereo / MS Stereo
Duration                                 : 1 min 11 s
Bit rate mode                            : Constant
Bit rate                                 : 128 kb/s
Channel(s)                               : 2 channels
Sampling rate                            : 44.1 kHz
Compression mode                         : Lossy
Writing library                          : LAME3.100
Encoding settings                        : -m j -V 4 -q 3 -lowpass 17 -b 128
```

Listing 5: MediaInfo output for audio created using OpenaI Jukebox

3.2.5 MOÛSAI

Schneider et al. (2023) provide Moûsai, which generates stereo music clips from a text prompt using a two-stage diffusion model. The first stage encodes the text prompt into an embedding then generates a compressed latent representation with a diffusion generator. The second stage uses another diffusion decoder to generate a waveform from the latent space.

Our team scraped 51 samples from the demonstration website for Moûsai[11] and 16 samples from the MusicGen website that were generated by Meta for comparison purposes. A representative MediaInfo output from one of the samples is shown in Listing 6. Of note is the duration of varying lengths, the sampling rate of 48 kHz, and the two channels.

[11] https://flavioschneider.notion.site/flavioschneider/Audio-Generation-with-Diffusion-c4f29f39048d4f03a23da13078a44cdb#e7075b29105f4d909d25b48c8f72863f

```
General
Complete name                            : 5
    _Death_Metal_2012_SEED384148608_CFG8.0.mp3
Format                                   : MPEG Audio
File size                                : 1.33 MiB
Duration                                 : 1 min 27 s
Overall bit rate mode                    : Constant
Overall bit rate                         : 128 kb/s
Writing library                          : LAME3.99.5

Audio
Format                                   : MPEG Audio
Format version                           : Version 1
Format profile                           : Layer 3
Format settings                          : Joint stereo / MS Stereo
Bit rate mode                            : Constant
Bit rate                                 : 128 kb/s
Channel(s)                               : 2 channels
Sampling rate                            : 48.0 kHz
Compression mode                         : Lossy
Writing library                          : LAME3.99.5
```

Listing 6: MediaInfo output for audio created using Moûsai

3.2.6 RIFFUSION

Forsgren & Martiros (2022) provide Riffusion, which is the Stable Diffusion (Rombach et al., 2022) model fine-tuned to generate images of spectrograms. The generated spectrograms are then converted to raw audio using the Griffin-Lim algorithm (Griffin & Lim, 1984), which estimates a signal given the magnitudes of its STFT, since magnitudes, but not phases, are the only information present in a spectrogram.

Our team scraped 13 Riffusion samples from the Riffusion website[12], as well as 15 Riffusion samples from the MusicGen website. A representative MediaInfo output from one of the samples is shown in Listing 7. Of note is the sampling rate of 32 kHz.

```
General
Complete name                            : 009_sample.mp3
Format                                   : MPEG Audio
File size                                : 941 KiB
Duration                                 : 30 s 60 ms
Overall bit rate mode                    : Constant
Overall bit rate                         : 256 kb/s
Writing library                          : LAME3.100

Audio
Format                                   : MPEG Audio
Format version                           : Version 1
Format profile                           : Layer 3
Duration                                 : 30 s 60 ms
Bit rate mode                            : Constant
Bit rate                                 : 256 kb/s
Channel(s)                               : 1 channel
Sampling rate                            : 32.0 kHz
Compression mode                         : Lossy
```

Listing 7: MediaInfo output for audio created using Riffusion

[12] https://www.riffusion.com

4 DATA SCIENCE PIPELINE

HARP follows the traditional data science pipeline as seen in Figure 8. As Data has already been covered in Section 3, this section will focus on Wrangling, Exploration, and Modeling.

Figure 8: HARP data science pipeline.

4.1 DATA WRANGLING

With raw audio files acquired from the various sources described in Section 3, the next step of data wrangling involved converting each raw audio file to have the same three PCM parameters. Therefore, each raw audio file is converted to have the following parameters:

- Sampling rate: 24 kHz
- Bit depth: 32-bit floating point
- Channels: 1

The sampling rate of 24 kHz was chosen because it is the lowest sampling rate among the data sources: MusicLM and Noise2Music samples are at this rate. This also has the added benefit of reducing memory and computational requirements of the model.

To speed up dataloading during training, all data sources were preprocessed into 24 kHz and 1 channel using the `ffmpeg` package[13]. Beyond those two parameters, when the `torchaudio` package loads a raw audio file into a `Tensor`, it inherently converts a 16-bit integer or 24-bit integer bit depth into a consistent 32-bit float representation. Then, once in `Tensor` form, any 2-channel stereo sources will be converted to 1-channel mono via computing the mean across the channels dimension.

Beyond the PCM encoding parameters, we want our data to have a consistent length of 5 seconds. Therefore, for any clip longer than 5 seconds, we trim the audio file to only be 5 seconds long. For any clip that is shorter than 5 seconds, the clip will be looped until it is 5 seconds long, i.e. until the `Tensor` has length dimension of 5 seconds $\times$ 24,000 samples/second $=$ 120,000 samples.

4.2 DATA EXPLORATION

One potential concern for our project is generalization. We used audio samples from a limited number of music generation models, and some of those models have relatively few audio samples. For example, we currently have over 7,000 samples from Jukebox, but only 28 samples from Riffusion, as previously shown in Table 2. Nonetheless, the distribution and diversity of the models across our dataset is an advantage in solving the generalization problem. More specifically, our dataset includes over 5,540 MusicGen audio files. Meta's MusicGen is widely considered to be the "go-to" state-of-the-art music generation model because of it's balance of accessibility and power. Of all models in out project, it is one of the most user-friendly to use and demonstrated results surpassing other existing model outputs. For this reason, our team ensured that the HARP training set was grounded on a larger set of MusicGen samples. With that said, our team also recognized that Google's MusicLM was likely the most powerful of the six chosen models. Unfortunately, these samples were limited to a total of 219. For this reason, we reserved all MusicLM samples specifically for the test set, as discussed later in Section 5.1, to help manage the critical issue of generalization.

[13] `https://www.ffmpeg.org/`

Beyond the diversity and distribution of the models in our project, we also examined the audio characteristics of the music generated by each model (see respective model sections in section 3.2). There is significant variation across the models in file duration, sampling rate, bit depth, channels (mono vs. stereo), etc. If fed directly into the model with these varying formats, it is likely that the model would learn features based on these external details. This was avoided by converting all files to have the same PCM parameters as discussed in Section 4.1.

4.3 Data Modeling

As described by Masood et al. (2021), the majority of audio deepfake detection techniques involve applying some form of neural network, often convolutional neural networks, to a spectral representation of the audio. These spectral representations are from a variety of modifications to the Fourier transforms, such as constant-Q transformation (CQT), linear frequency cepstrum[14] coefficients (LFCC), or mel-spectrograms. The mel-spectrogram is a spectrogram where the frequency axis has been converted from a linear scale to a logarithmic "mel scale" such that the resulting frequencies are perceptually equidistant to a listener. The following formula converts frequency in hertz into mels:

$$m = 2595 \log_{10} \left(1 + \frac{f}{700} \right) \tag{2}$$

4.3.1 Proposed Model

One particular example is utilizing transfer learning, in which a model that has been trained on one task is re-applied for a separate task. Rahul et al. (2020) applied this concept by taking the mel-spectrogram of suspected spoofed speech audio and feeding the resultant image through a ResNet34 (He et al., 2015) pre-trained on image classification. Their experimental results utilizing transfer learning on mel-spectrograms of the Automatic Speaker Verification (ASV) Spoof 2019 dataset outperformed the baseline results that utilized CQTs or LFCCs. This was the base model our team started with for `HARP`, but repurposed for musical audio rather than speech.

After further experimentation and literature review, our team changed course, adopting a much different model pipeline. After converting raw audio into the mel-spectrogram, harmonic-percussive source separation (HPSS) median filtering is applied (discussed further in Section 4.3.2). After HPSS, the original mel-spectrogram is now two distinct spectrograms, each representing a different part of the signal (either harmonic or percussive). Our most critical departure from the base model mentioned above, was our choice to use a vision transformer (ViT) in lieu of a convolutional neural network like ResNet34 (both of these are discussed previously in Sections 2.1 and 2.2). Upon ending at a dense fully connected layer, the model's final output is its binary classification of the audio as either human-generated or AI-generated. Figure 9 summarizes the final `HARP` model pipeline.

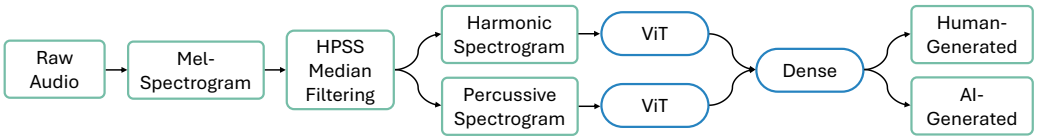

Figure 9: HARP model pipeline

4.3.2 Data Augmentation

Since new models or methods to create AI-generated music can continue to be developed, the generalization and robustness of `HARP` is of particular concern. For CNN applications, Yin et al. (2019) note that, as long as a diverse augmentation space is defined, methods such as AutoAugment "[achieve] state-of-the-art robustness." AutoAugment, from Cubuk et al. (2019a), is a method to automatically search for "improved data augmentation policies." Since then, Cubuk et al. (2019b)

[14]The cepstrum is the inverse Fourier transform of the logarithm of the power spectrum

introduced RandAugment, and Müller & Hutter (2021) introduced TrivialAugment, which both outperform AutoAugment.

Therefore, to increase the robustness of `HARP`, we applyied TrivialAugment, as detailed in Algorithm 1. During training, TrivialAugment randomly selects an augmentation method and strength to apply to the raw audio before it is convered to spectogram. To achieve this, we define a custom and diverse augmentation space designed specifically for raw audio. Table 3 lists the audio-specific transformation that make up the augmentation set $\mathbb{A}$, along with the transformation parameter that will correspond with augmentation strength m. Examples of these augmentations can be heard on the PyTorch website[15].

Algorithm 1: Modified TrivialAugment for audio

input : A mono raw audio vector $\boldsymbol{x}$, of length s
output: Augmented raw audio vector $\boldsymbol{x}'$, of length s

1. Sample an augmentation a from $\mathbb{A}$
2. Sample a strength m from $[0, 1)$
3. Return $\boldsymbol{x}' = a(\boldsymbol{x}, m)$

To additionally increase resilience to perturbations, our team evaluated the use of a form a median filtering that results in harmonic-percussive source separation (FitzGerald, 2010). The median filtering decomposes the original spectrum into harmonic and percussive components, and we then apply parallel models to these component spectrums before reconnecting in the fully connected layer. Figure 10 visualizes the results of this median filtering.

Table 3: List of transformations in the audio augmentation set

Transformation	**Strength Parameter**
Lowpass Filter	Cutoff Frequency
Highpass Filter	Cutoff Frequency
Speed Reduction	Final speed
Speed Increase	Final speed
Reverb	Amount of reverb added
Add Gaussian noise	Signal-to-noise ratio (SNR)
Time Mask	Fraction of audio silenced
Distortion	Percentile of samples to clip
Apply MP3 compression	Bitrate
Apply Vorbis compression	Bitrate
Identity	–

[15] `https://pytorch.org/audio/stable/tutorials/audio_data_augmentation_tutorial.html`

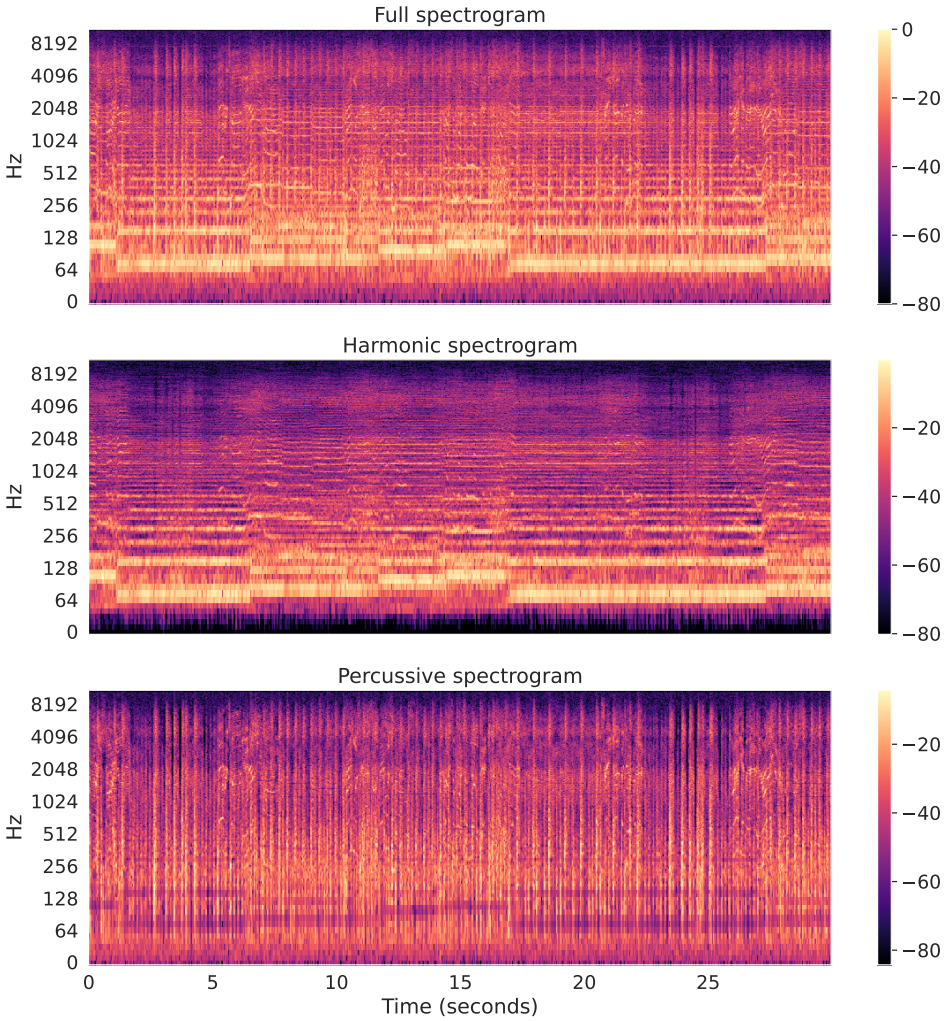

Figure 10: The original spectogram of a 30 second music clip, along with the harmonic and percussive decompositions

5 EXPERIMENTS

5.1 SETUP

In order to train and test our model, we split our dataset into training, validation, and testing sets. Overall, we used a 60/20/20 split into train, validation, and test respectively. In additional to the standard split, we implemented a form of "leave-one-out" testing, whereby an entire music generation model is held out solely for testing. All samples from the MusicLM model were held out specifically for testing in order to evaluate whether our classification model can generalize to a previously unseen data generation mechanism. Additionally, we reduced the number of Jukebox samples used from 7,100 to 2,027 in order to ensure an equal distribution of human- and AI-generated samples across all three splits. Table 4 summarizes the overall split based on our current data set.

Table 4: Distribution of Train-Validation-Test Splits

Model	**Train**	**Validation**	**Test**
Free Music Archive	4,800	1,600	1,600
MusicGen	3,391	1,150	999
MusicLM**	0	0	219
Noise2Music	73	23	23
Jukebox	1,279	408	340
Moûsai	41	13	13
Riffusion	16	6	6
Total	9,600 (60%)	3,200 (20%)	3,200 (20%)

***MusicLM is reserved exclusively for Test set*

The inspiration for our model comes from Dessa's DeepFake Audio Detection Github repository[16], which contains the open source code for their model to detect AI-generated speech files. Although Dessa utilized a diverse set of models for training, validation, and testing, they too reserved one model specifically for only testing, just as our team has done with MusicLM.

To define our base model, we extend a PyTorch LightningModule[17]. To take advantage of transfer learning, the model then loads either a pre-trained ResNet152[18] or a pre-trained ViT Large (16x16 patch size)[19] from the `torchvision` library. The final dense layer of the pre-trained model is reset and modified to have a single logits output for binary classification, then all other weights except the dense layer are frozen.

To implement the HPSS-based version of our model, we simply load two pre-trained ResNet or ViT modules, one for the harmonic spectrogram and one for the percussive spectrogram. The last dense layer of each network is modified to have a hyperparameter-specified output size. Then, those two output vectors are concatenated and fed through a final dense layer to a single logits output.

Based on this, a forward pass through the model consists of the following steps:

1. Convert the raw audio into spectrogram representation
2. If specified, convert the spectrogram from linear frequency to mel scale
3. If specified, apply HPSS median filtering to split the spectrogram into harmonic and percussive spectrograms
4. Convert the spectrogram(s) into a color image using a hyperparameter-defined colormap
5. Pass the spectrogram(s) through the ResNet or ViT typical forward pass

[16] `https://github.com/dessa-oss/fake-voice-detection`
[17] `https://lightning.ai/docs/pytorch/stable/common/lightning_module.html`
[18] `https://pytorch.org/vision/main/models/resnet.html`
[19] `https://pytorch.org/vision/main/models/vision_transformer.html`

To train this model, we utilize the binary cross entropy loss via the PyTorch BCEWithLogitsLoss[20] class for numerical stability. The optimizer used during trainer is the the synergistic Ranger21 optimizer from Wright & Demeure (2021), which combines the Adam optimizer with several improvements, such as gradient clipping to ensure the gradient does not get overly destabilized by certain mini-batches that happen to have high loss. Then, to validate the model, we compute the classification accuracy for the validation set:

$$\text{Accuracy} = \frac{1}{N}\sum_{i}^{N} \mathbb{I}(y_i = \hat{y}_i) \tag{3}$$

where y_i is the ground truth classification and $\hat{y}_i$ is the predicted class.

The hyperparameters for our model are described in Table 5. Hyperparameter tuning was performed using Bayesian optimization. Specifically, we used Tree-structured Parzen Estimator (TPE) optimization (Watanabe, 2023), using the Optuna optimization framework (Akiba et al., 2019). Each hyperparameter combination suggested by TPE was trained for 10 epochs, then validation set accuracy was used as the performance metric for the trial. To save on resources, any trial whose validation accuracy was below the median within the first few epochs was pruned. Table 6 shows the best hyperparameters found during optimization.

Table 5: List of hyperparameters for the HARP model

Hyperparameter	**Description**
Transfer learning model	Whether to use ResNet or ViT
Learning rate	The learning rate for the optimizer
n_fft	The size of the Fast Fourier Transform (FFT) to use in the spectrogram transform
Colormap	The name of the `matplotlib` colormap[21] to use, such as "magma"
Mel scale	Whether to convert frequencies to the mel scale
Spectogram decibel (dB) cutoff	The minimum negative cut-off for the spectrogram, i.e. any quieter than this dB value will be colored black
HPSS	Whether to use the HPSS-based model
HPSS Layer Size	If using the HPSS-based model, the input size of the last dense layer

Table 6: Post-optimization hyperparameters

Hyperparameter	**Value**
Transfer learning model	ViT Large 16x16
Learning rate	0.00109
n_fft	2048
Colormap	Magma
Mel scale	True
Spectogram decibel (dB) cutoff	60 dB
HPSS	True
HPSS Layer Size	921

[20] `https://pytorch.org/docs/stable/generated/torch.nn.BCEWithLogitsLoss.html`

5.2 Results

We trained the model using the optimal hyperparameters for a maximum of 250 epochs. In order to avoid overfitting and maximize generalization, we applied early stopping if the validation accuracy did not improve for 5 epochs. Ultimately, the training was stopped early after 122 epochs, with a validation accuracy of 84.9%. Given that our goal is a binary classification task, for testing we use some of the classic metrics for identifying binary classification performance, including precision, recall, and the receiver operating characteristic (ROC) curve and the area under the ROC curve (AUROC). Table 7 shows these metrics computed for the validation set predictions and the testing set predictions. Figure 11 shows the ROC curve for both the validation dataset and the test dataset.

Table 7: Final validation set and test set metrics

Metric	Validation Set	Testing Set
Accuracy	84.90%	81.84%
F1 Score	85.20%	81.94%
Precision	83.58%	81.51%
Recall	86.87%	82.38%
AUROC	0.922	0.901

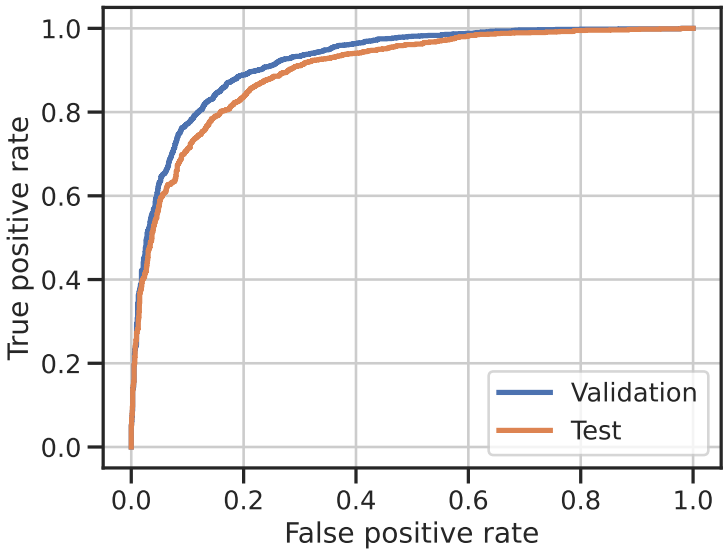

Figure 11: Receiver operating characteristic curve for HARP

Notably, the model performs similarly well between the two sets of unseen data, even considering that the test dataset contains an entirely unseen AI-music model in the form of MusicLM. Recall appears to be the metric that suffered the most with the addition of the MusicLM data, implying that there are likely AI-generated samples that escaped detection.

6 Conclusions

6.1 Impact

As mentioned in our problem statement, the advent of generative AI models is a real threat to artistic works. Future concerns include the theft of protected creations, such as famous artists being under the threat of unlicensed deep-fake vocal audio or audio directly sampled from original compositions. Additionally, the use of artificially generated content will slowly begin to erode the natural creative

process of original music. Because these models are trained on existing works, they may struggle to produce innovative content–the kind of genre defining creativity as seen in the likes of jazz, rock, hip-hop, etc. This could be viewed as an overarching threat to music itself. If AI-generated music proliferates to the point that it becomes a significant part of the musical canon, it may cannibalize itself. In 2023, Shumailov et al. (2023) found that machine learning models trained on model-generated content "causes irreversible defects in the resulting models". They found that this corruption applied to variational autoencoders, Gaussian mixture models, and LLMs, which are the basis for many of the music generation models. If human creators stop producing music, the AI models will likely be unable to continue for long.

The problem is that artists, record labels, streaming services, etc. need an effective means to combat music generated through artificial intelligence techniques. Manual verification is not only time consuming but probably even relatively ineffective. The best strategy in preventing harmful, negative impacts from artificially generated music is the use of a music detection model like `HARP`, which can successfully distinguish human music from AI music nearly 82% of the time. As generative AI models continue to grow stronger, the imperative to create and improve upon AI detection models like `HARP` becomes ever more pressing.

6.2 Future Work

Although our team is satisfied with the overall performance of `HARP`, we recognize that there are potential limitations and therefore improvement opportunities. The overall architecture of the `HARP` model, including the use of mel-spectrograms, HPSS median filtering, vision transformers, etc., is sound. Regardless, we believe that diversity and total count of the dataset could be a key differentiator in improving future performance. With greater access to the music models, such as increased sampling from MusicLM, Noise2Music, Mousai, and Riffusion (all of which were underrepresented in our `HARP` dataset realtive to MusicGen and Jukebox) and the release of future improved models, `HARP` could be trained, validated, and tested on a more diverse and comprehensive dataset, potentially enhancing both its performance and generalization in detecting various AI-generated audio samples, irrespective of the underlying models that created them. Our hope is that `HARP` can serve as a valuable contribution in the open-source AI-detection model space, providing aspiring data scientists with a building block for the future.

References

Andrea Agostinelli, Timo I. Denk, Zalán Borsos, Jesse Engel, Mauro Verzetti, Antoine Caillon, Qingqing Huang, Aren Jansen, Adam Roberts, Marco Tagliasacchi, Matt Sharifi, Neil Zeghidour, and Christian Frank. Musiclm: Generating music from text, Jan 2023.

Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework, 2019.

Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. 2019.

Jade Copet, Felix Kreuk, Itai Gat, Tal Remez, David Kant, Gabriel Synnaeve, Yossi Adi, and Alexandre Défossez. Simple and controllable music generation, Jun 2023.

Ekin D. Cubuk, Barret Zoph, Dandelion Mane, Vijay Vasudevan, and Quoc V. Le. Autoaugment: Learning augmentation policies from data, Apr 2019a.

Ekin D. Cubuk, Barret Zoph, Jonathon Shlens, and Quoc V. Le. Randaugment: Practical automated data augmentation with a reduced search space, Nov 2019b.

Michaël Defferrard, Kirell Benzi, Pierre Vandergheynst, and Xavier Bresson. FMA: A Dataset For Music Analysis, 2017.

Dessa. Fake voice detection, 2020. URL `https://github.com/dessa-oss/fake-voice-detection`.

Prafulla Dhariwal, Heewoo Jun, Christine Payne, Jong Wook Kim, Alec Radford, and Ilya Sutskever. Jukebox: A generative model for music, 2020.

Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale, 2020.

Alexandre Défossez, Jade Copet, Gabriel Synnaeve, and Yossi Adi. High fidelity neural audio compression, Oct 2022.

Derry FitzGerald. Harmonic/Percussive Separation Using Median Filtering. In *Proc. of the 13th Int. Conference on Digital Audio Effects*, Sep 2010.

Seth Forsgren and Hayk Martiros. Riffusion - Stable diffusion for real-time music generation, 2022. URL `https://riffusion.com/about`.

Daniella Genovese. John legend calls for regulation of ai-generated music, 2023. URL `https://www.foxbusiness.com/lifestyle/john-legend-calls-regulation-ai-generated-music`. Accessed: 2023-06-30.

D. Griffin and Jae Lim. Signal estimation from modified short-time fourier transform. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 32(2):236–243, 1984. doi: 10.1109/TASSP.1984.1164317.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition, 2015.

Qingqing Huang, Daniel S. Park, Tao Wang, Timo I. Denk, Andy Ly, Nanxin Chen, Zhengdong Zhang, Zhishuai Zhang, Jiahui Yu, Christian Frank, Jesse Engel, Quoc V. Le, William Chan, Zhifeng Chen, and Wei Han. Noise2music: Text-conditioned music generation with diffusion models, Mar 2023.

Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. In F. Pereira, C.J. Burges, L. Bottou, and K.Q. Weinberger (eds.), *Advances in Neural Information Processing Systems*, volume 25. Curran Associates, Inc., 2012. URL `https://proceedings.neurips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf`.

Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel. Backpropagation Applied to Handwritten Zip Code Recognition. *Neural Computation*, 1(4):541–551, 12 1989. ISSN 0899-7667. doi: 10.1162/neco.1989.1.4.541. URL `https://doi.org/10.1162/neco.1989.1.4.541`.

Yann Lecun, Larry Jackel, Corinna Cortes, John Denker, Harris Drucker, Isabelle Guyon, Urs Muller, Eduard Sackinger, Patrice Simard, and Vladimir Vapnik. Learning algorithms for classification: A comparison on handwritten digit recognition. *The Statistical Mechanics Perspective*, 07 1995.

Momina Masood, Marriam Nawaz, Khalid Mahmood Malik, Ali Javed, and Aun Irtaza. Deepfakes Generation and Detection: State-of-the-art, open challenges, countermeasures, and way forward, Nov 2021.

Christopher Montgomery. A digital media primer for geeks. `https://xiph.org/video/vid1.shtml`, 2010. Accessed June 2023.

Samuel G. Müller and Frank Hutter. Trivialaugment: Tuning-free yet state-of-the-art data augmentation, Aug 2021.

Faustine Ngila. Spotify, apple music removed ai-generated music over copyright claims: Report, 2023. URL `https://finance.yahoo.com/news/spotify-apple-music-removed-ai-153447147.html`. Accessed: 2023-06-30.

T P Rahul, P R Aravind, Ranjith C, Usamath Nechiyil, and Nandakumar Paramparambath. Audio Spoofing Verification using Deep Convolutional Neural Networks by Transfer Learning, Aug 2020.

Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 10684–10695, June 2022.

Flavio Schneider, Zhijing Jin, and Bernhard Schölkopf. Moûsai: Text-to-Music Generation with Long-Context Latent Diffusion, 2023.

Ilia Shumailov, Zakhar Shumaylov, Yiren Zhao, Yarin Gal, Nicolas Papernot, and Ross Anderson. The curse of recursion: Training on generated data makes models forget, 2023.

Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning, May 2018.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2017.

Shuhei Watanabe. Tree-structured parzen estimator: Understanding its algorithm components and their roles for better empirical performance, 2023.

Less Wright and Nestor Demeure. Ranger21: a synergistic deep learning optimizer, 2021.

Dong Yin, Raphael Gontijo Lopes, Jon Shlens, Ekin Dogus Cubuk, and Justin Gilmer. A fourier perspective on model robustness in computer vision. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett (eds.), *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. URL https://proceedings.neurips.cc/paper_files/paper/2019/file/b05b57f6add810d3b7490866d74c0053-Paper.pdf.

Neil Zeghidour, Alejandro Luebs, Ahmed Omran, Jan Skoglund, and Marco Tagliasacchi. Soundstream: An end-to-end neural audio codec, Jul 2021.